

# **PAC2: Design and architecture**

## Performance analysis of social network models in agent based systems

Martí Gimeno Ortí

Universitat Oberta de Catalunya. 2025-2026



# Table of Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                          | <b>7</b>  |
| 1.1 Problem description                                        | 7         |
| 1.2 State of the art                                           | 7         |
| 1.3 Objectives                                                 | 8         |
| 1.4 Methodology                                                | 9         |
| 1.5 Planning                                                   | 9         |
| 1.6 Ethical, social and environmental impact                   | 10        |
| <b>2 Model descriptions</b>                                    | <b>11</b> |
| 2.1 Misinformation diffusion model                             | 11        |
| 2.1.1 Epidemiological Foundations: The Baseline SBFC Framework | 12        |
| 2.1.2 Architectural Enhancements: Heterogeneous Agent Classes  | 15        |
| 2.1.3 Experimental Input Parameters                            | 16        |
| 2.1.4 Validation Methodology                                   | 17        |
| <b>3 Model implementations</b>                                 | <b>19</b> |
| 3.1 Parallelizing the misinformation diffusion model           | 19        |
| <b>Bibliography</b>                                            | <b>23</b> |

## List of Figures

|          |                                                              |    |
|----------|--------------------------------------------------------------|----|
| Figure 1 | States and transitions of the SBFC model .....               | 13 |
| Figure 2 | Repast HPC memory management applied to the SBFC model ..... | 22 |

## List of Tables

|         |                                                                  |    |
|---------|------------------------------------------------------------------|----|
| Table 1 | Gantt diagram for task planning .....                            | 10 |
| Table 2 | Agent type characteristics of the heterogeneous SBFC model ..... | 16 |
| Table 3 | Input parameters of the heterogeneous SBFC model .....           | 17 |



# Chapter 1

## Introduction

### 1.1 Problem description

Agent Based Modelling is a computational method used to simulate the actions and interactions of autonomous agents (individuals, companies, cells) and study their effects on the system as a whole. Its goal is to understand complex phenomena by watching it emerge from the individual interactions between a large number of agents. The complex group behavior is not modelled nor programmed directly, but it naturally occurs, as in an ant colony.

High Performance Computing (HPC) is the practice of aggregating computing power in a way that delivers much higher performance than a single workstation could deliver. It relies on the concept of parallel processing in order to break down a single job with considerable computational work into multiple smaller jobs that can be processed at the same time. All HPC systems are composed of nodes, which perform the calculations, and interconnections, which allow for communication between nodes.

Given the parallel nature of agent based systems, it's logical to combine the two disciplines to unlock the ability to simulate millions of agents in reasonable time. But, can one quantify the real performance gain of implementing an agent based model in a high performance computing platform? Which are the performance bottlenecks of the HPC platforms that accelerate ABMS? Is there any way to alleviate those? This Master thesis work will answer the mentioned questions by studying the performance of Agent Based systems implemented in High Performance Computing platforms.

### 1.2 State of the art

Over the past two decades, several frameworks for implementing agent-based modeling and simulation (ABMS) on high-performance computing (HPC) systems have been proposed and applied to real-world case studies. Among the most prominent examples are FLAME [1], FLAME GPU [2], RepastHPC [3], [4] and DMASON [5]. These frameworks are designed to allow domain specialists to build large-scale simulations without directly dealing with the low-level complexities of parallelism. Despite this shared motivation, they adopt different design choices: agents and environments may be defined using diverse languages and formalisms, inter-agent interactions can rely on distinct communication strategies, and different support for features such as load balancing. These architectural and implementation choices influence scalability and

efficiency, leading to substantially different performance outcomes depending on the framework being chosen and the simulation model characteristics.

Benchmarks on the performance discrepancies between different HPC frameworks and ABMS characteristics have been performed by [6] and [7], shedding light on the problem of which HPC framework to select depending on the simulation model. This work focuses in the RepastHPC framework to implement the parallelized versions of the agent based models. It is selected for its C++ based architecture designed specifically for TOP500 class supercomputers, utilizing MPI (Message Passing Interface) for efficient cross-process communication and spatial decomposition.

The Computational Model Library provided by the CoMSES Net [8], [9] has been the primary database for exploring and publishing ABMS since its publication more than a decade ago. This platform acts as a common sharing ground of peer-reviewed, reproducible models that follow the standard ODD protocol, providing a reliable foundation for the performance analysis.

### 1.3 Objectives

The main objective of the work is to evaluate the performance and identify the bottlenecks of two existing complex agent based models implemented in the Repast HPC framework. To achieve that, the following intermediate objectives are presented:

1. **Models research and selection:** Selecting two representative social network agent based models, that could be implemented in an HPC framework to support millions of agents.
2. **Detailed analysis for the selected models:** The exhaustive analysis will include descriptions on the model internal logic, the agent relationship and communication, the complexity of the agent and environment logic and the inputs of outputs of the model.
3. **Implementing the selected ABMS in an HPC framework:** Description of the Repast HPC framework fundamentals and how each model has to be implemented according to Repast HPC concepts like the Scheduler, the Context and MPI communication. Validation and verification of the programs to ensure that the model has been implemented properly.
4. **Simulation execution and analysis performance:** Benchmark executions with parameter sweeping and scalability tests to measure performance metrics like execution time, resource usage, and parallel efficiency.
5. **Bottleneck identification:** The main part of the analysis will be centered around identifying, quantifying and describing the nature of the inefficiencies and bottlenecks of each model.

Finally, a secondary objective on a **graph partitioning algorithm benchmarking** focuses on testing a new load balancing algorithm that distributes work between nodes



in an HPC cluster by partitioning a graph in a certain new way to improve the overall system performance.

## 1.4 Methodology

To achieve the mentioned objectives in the previous section, the following methodology will be applied

- **ABMS literature review:** Use CoMSES Net model repository and academic databases to analyse and select two social networks agent based models. In order to validate the model selection, the following properties will be analyzed:
  - *Model domain:* Description of the model representation, inputs, outputs and purpose.
  - *Agent relationship:* Number of agent kinds and the relationship between them, description of their communication mechanism, content and criteria.
  - *Model complexity:* Evaluation of the underlying logic for agents and environment
- **Implementing the ABMS in an HPC framework:** Select the appropriate framework using the benchmarks designed for ABMS performance evaluation for HPC in [7] and [10].
- **Define testing hardware, environment and performance metrics:** The specific HPC infrastructure for executing the models is currently under evaluation. The final selection will depend on resource availability and the specific hardware requirements dictated by the complexity of the selected models.

## 1.5 Planning

Follows an enumeration and description of all the tasks needed to achieve the work's objectives:

1. **ABMS research and selection**
  - 1.1. Introduction to CoMSES Net's Computational Model Library.
  - 1.2. Preselection of most interesting ABMS found with an initial superficial analysis.
  - 1.3. Final selection of 2 ABMS that fit the criteria defined previously.
2. **Models in-depth analysis**
  - 2.1. Description of the model domain, inputs, outputs and purpose.
  - 2.2. Description of the model agents behaviour, relationship and parameters.
  - 2.3. Identification of the parameter sweeping for performance evaluation.
3. **HPC implementation**
  - 3.1. Introduction to HPC frameworks and their unique characteristics.
  - 3.2. Introduction to RepastHPC.
  - 3.3. Implementation of models with RepastHPC.
4. **Execution and validation**
  - 4.1. Programs validation to ensure that the models have been implemented properly.

- 4.2. Executions with parameter sweeping and scalability tests to measure performance metrics: execution time, resource usage, parallel efficiency...

## 5. Performance analysis

- 5.1. Analysis of performance metrics results.  
5.2. Identification of bottleneck and possible causes.

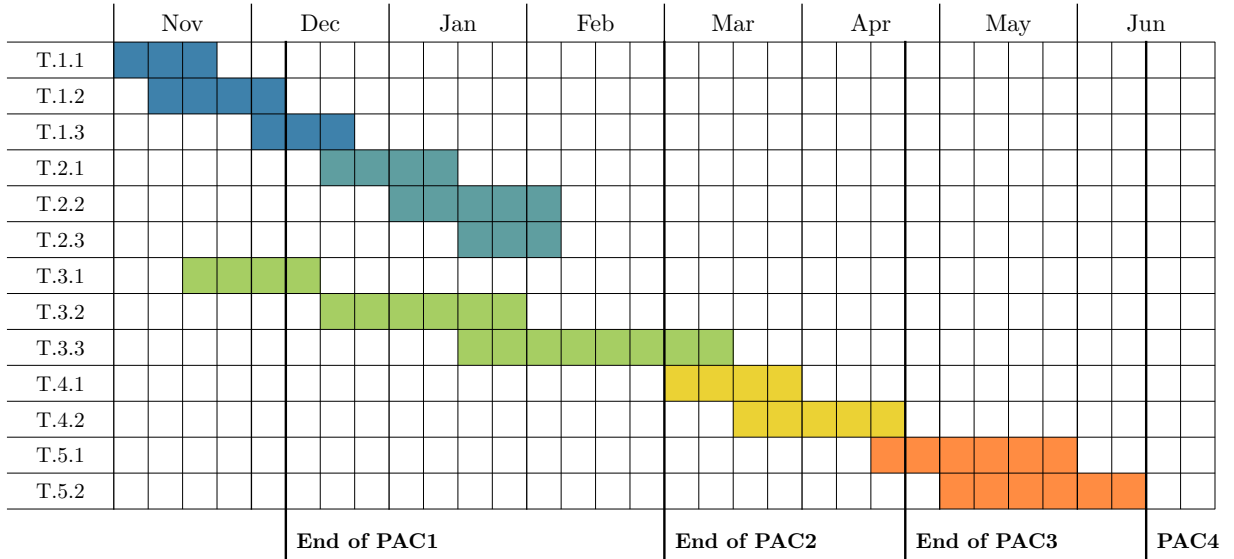


Table 1: Gantt diagram for task planning

## 1.6 Ethical, social and environmental impact

This work contributes to the advancement of computational social science by enabling larger and more accurate simulations of complex systems. By identifying performance bottlenecks in economic and social network models, this research facilitates the creation of digital twins that can be used by governments to predict economic crises or the spread of misinformation with higher fidelity.

Ethically, the work promotes transparency in algorithmic decision-making by optimizing open-source tools like RepastHPC and relying on open-access model libraries like CoMSES, democratizing access to powerful simulation capabilities.

Environmentally, the focus on performance optimization and bottleneck identification directly relates to energy efficiency. By reducing the computational resources required to run massive simulations, the carbon footprint of large-scale research projects can be minimized.

## Chapter 2

# Model descriptions

The evaluation of High Performance Computing frameworks for Agent-Based Modelling and Simulation requires the selection of baseline models that are robust, computationally significant, and highly relevant to contemporary science. Following the methodology and objectives outlined in the introductory chapter, the models selected for this performance analysis operate within the domain of computational social science, specifically focusing on the diffusion of information in a social network. This chapter provides an exhaustive analysis of the models internal mechanisms, mathematical foundations, agent interactions, and the specific variables that will govern the subsequent HPC implementation and benchmarking phases.

### 2.1 Misinformation diffusion model

The first selected model, archived in the COMSES Net Computational Model Library under version 1.0.0, is titled “*Controlling the Misinformation Diffusion in Social Media by the Effect of Different Classes of Agents*” [11], [12] and describes the spread of misinformation and the corresponding countermeasures in digital social networks.

This model represents a sophisticated architectural extension of the foundational Susceptible-Believer-FactChecker (SBFC) epidemic model originally proposed by Tambuscio et al. [13], [14]. While the original SBFC framework established the mathematical bedrock for simulating the competitive diffusion of a viral hoax and its debunking information within a homogeneous population, the selected extended model introduces significant topological and behavioral complexities that mirror real-world dynamics. By categorizing the network nodes into distinct, heterogeneous agent types: Scholars, Influencers, Normal Agents, and Bots, the model accurately captures the asymmetric influence and dynamic belief structures present in real-world platforms like Facebook [15].

From a computational perspective, this selected model serves as an ideal stress-test for HPC performance analysis. The introduction of heterogeneous agent classes and targeted intervention strategies significantly increases the computational load required for each agent evaluation cycle. Furthermore, simulating this environment over a large-scale, highly connected graph introduces intricate memory access patterns and inter-agent communication dependencies that are notoriously difficult to parallelize efficiently. These characteristics are precisely what expose the performance bottlenecks, parallelization inefficiencies, and synchronization overheads inherent in distributed-memory platforms such as Repast HPC.

### 2.1.1 Epidemiological Foundations: The Baseline SBFC Framework

The Susceptible-Believer-FactChecker model conceptualizes the spread of misinformation not as a simple, unidirectional contagion, but as a complex, simultaneous competitive process between a virus (the hoax) and its cure (the debunking information). By modeling hoaxes analogously to biological viruses, the framework borrows heavily from traditional compartmental epidemic models such as the Susceptible-Infected-Recovered (SIR) and Susceptible-Infected-Susceptible (SIS) paradigms. However, it splits the “Infected” compartment into two distinct and adversarial sub-compartments, creating a much more dynamic state space.

#### I. Agent State Space and Network Topology

The simulation environment is mathematically represented as a network graph  $G = (V, E)$ , where  $V$  represents the set of vertices (the individual agents or users) and  $E$  represents the set of edges (the social connections or follower relationships between them). At any discrete time step  $t$ , each agent is characterized by a specific behavioral state: This state is rigorously defined by a triple of binary indicators:

$$\forall i \in V \quad s_{i(t)} = [s_i^B(t), s_i^F(t), s_i^S(t)] = \begin{cases} [1, 0, 0] \\ [0, 1, 0] \\ [0, 0, 1] \end{cases} \quad (1)$$

The state vector is constrained such that it can only assume one of three mutually exclusive configurations, corresponding to the agent’s current cognitive stance regarding the circulating information:

- **Believer (B)**: Represented as  $s_i^B(t) = 1$ . In this state, the agent  $i$  has been exposed to the hoax, believes it to be true, and actively participates in its propagation across the network.
- **Fact-Checker (FC)**: Represented as  $s_i^F(t) = 1$ . In this state, the agent  $i$  has either independently verified the news or been exposed to debunking information. The agent recognizes the information as a hoax, adopts the factual narrative, and actively propagates the truth to its neighbours.
- **Susceptible (S)**: Represented as  $s_i^S(t) = 1$ . This is a neutral state. The agent  $i$  is either entirely ignorant of the circulating narrative, or they have forgotten a previously held belief (either belief in the hoax or belief in the facts).

#### II. Stochastic Transition Dynamics

The temporal evolution of the system is governed by a stochastic, discrete-time Markov process. The probability that an individual agent  $i$  transitions into or remains in a specific state at time  $t + 1$  is defined by a probability mass function:

$$\begin{aligned}
p_i(t+1) &= [p_i^B(t), p_i^F(t), p_i^S(t)] = \\
&= \begin{cases} p_i^B(t+1) = f_i s_i^S(t) + (1 - p_{\text{forget}} - p_{\text{verify}}) s_i^B(t) \\ p_i^F(t+1) = g_i s_i^S(t) + p_{\text{verify}} s_i^B(t) + (1 - p_{\text{forget}}) s_i^F(t) \\ p_i^S(t+1) = p_{\text{forget}} (s_i^B(t) + s_i^F(t)) + (1 - f_i - g_i) s_i^S(t) \end{cases} \quad (2)
\end{aligned}$$

where  $p_{\text{forget}}$ ,  $p_{\text{verify}}$  are constant probabilities and  $f_i$ ,  $g_i$  are the spreading functions that provide the network effect. Figure 1 shows the three states and their primary sociocognitive phenomena that drive these probability distributions and subsequent transitions: Forgetting ( $p_{\text{forget}}$ ), Verifying ( $p_{\text{verify}}$ ) and Spreading ( $f_i$ ,  $g_i$ ).

The actual, deterministic state of the agent at the next sequential time step is then determined by a random realization of this distribution:

$$s_i(t+1) = \text{MultiRealize}[p_i(t+1)]. \quad (3)$$

### III. The Spreading Mechanism

Two continuous spreading functions,  $f_i(t)$  and  $g_i(t)$ , calculate the exact probability of a Susceptible agent  $i$  adopting the hoax or adopting the debunking information, respectively. These functions evaluate the ratio of Believers and Fact-Checkers within the agent's immediate network neighborhood, modulated by global system parameters:

$$f_i(t) = \beta \frac{n_i^B(t) \cdot (1 + \alpha)}{n_i^B(t) \cdot (1 + \alpha) + n_i^F(t) \cdot (1 - \alpha)} \quad (4)$$

$$g_i(t) = \beta \frac{n_i^F(t) \cdot (1 - \alpha)}{n_i^B(t) \cdot (1 + \alpha) + n_i^F(t) \cdot (1 - \alpha)} \quad (5)$$

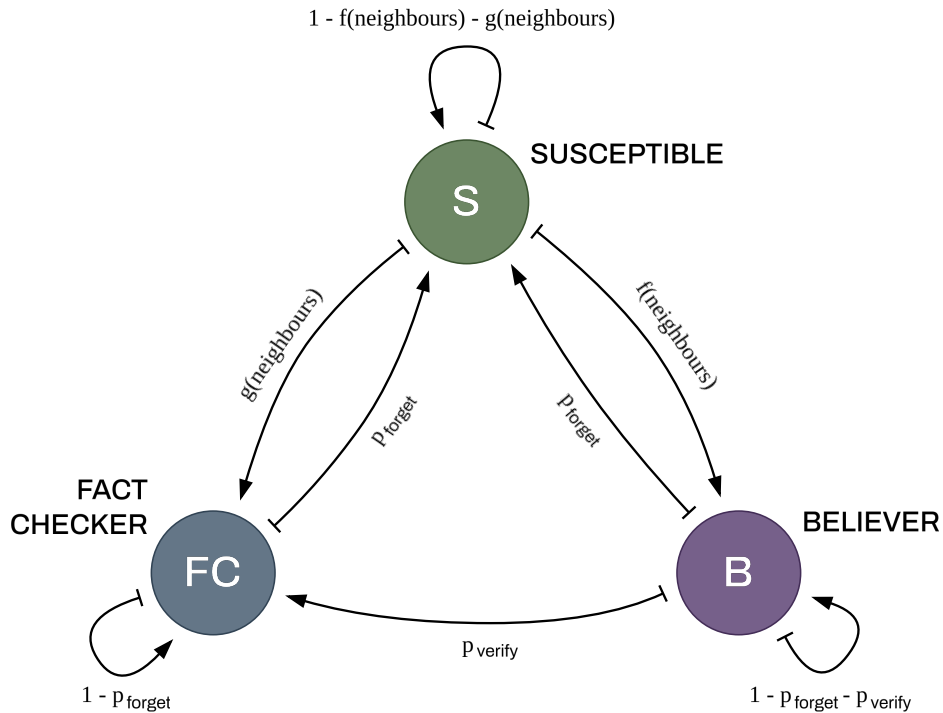


Figure 1: States and transitions of the SBFC model

In these formulations,  $n_i^B(t)$  and  $n_i^F(t)$  represent the absolute, integer count of agent  $i$ 's direct neighbours who are currently in the Believer and Fact-Checker states at time  $t$ . The parameter  $\beta \in [0, 1]$  represents the base spreading rate, defining the overall transmissibility or virality of information across the network, independent of its truthfulness. The parameter  $\alpha \in [0, 1]$  is a constant parameter for the credibility of the hoax. It is critical to note that  $f_i(t) + g_i(t) = b\eta$ , which aligns the total aggregate infection rate with traditional SIS epidemic models, ensuring mathematical consistency.

From a high-performance computing perspective, the evaluation of  $n_i^B(t)$  and  $n_i^F(t)$  represents one of the most significant computational burdens in the simulation. For an agent to calculate its spreading functions, the processor must fetch the state data of every single vertex connected to agent  $i$ . In dense, scale-free networks, a single agent may possess thousands of edges, requiring massive, non-contiguous memory access operations that frequently result in cache misses and degraded performance.

#### IV. Results

The original authors of the SBFC model conducted extensive mean-field approximations to understand the infinite-time limit behavior of the system, assuming a homogeneous network where all vertices have the same degree  $k$ . Their mathematical analysis demonstrated that the density of susceptible individuals at equilibrium ( $S_\infty$ ) depends solely on the spreading rate  $\beta$  and the forgetting probability  $p_{\text{forget}}$ , calculated as:

$$S_\infty = \frac{p_{\text{forget}}}{\beta + p_{\text{forget}}} \quad (6)$$

The core analytical breakthrough of the baseline model, however, was the derivation of a hard threshold for the verification probability that guarantees the complete eradication of the hoax from the network, achieving a state where  $B_\infty = 0$ . By setting the equilibrium equation for Believers to zero and solving for  $p_{\text{verify}}$ , the sufficient condition for hoax eradication is expressed as:

$$p_{\text{verify}} \geq \frac{2\alpha}{1 - \alpha} \cdot p_{\text{forget}} \quad (7)$$

This critical equation establishes that the minimum required fact-checking effort to eradicate a hoax is not linear. Instead, it scales non-linearly with the hoax's inherent credibility  $\alpha$  and proportionally with the network's overall forgetting rate. If a hoax is highly credible ( $\alpha$  approaches 1), the required verification effort approaches infinity. While this elegant mathematical threshold holds true for homogeneous, mean-field approximations, the dynamics alter significantly, and become vastly more difficult to predict, when network topologies exhibit real-world scale-free properties and when agent behaviors become heterogeneous. This limitation in the baseline model necessitates the transition to the extended model selected for this thesis.

### 2.1.2 Architectural Enhancements: Heterogeneous Agent Classes

The fundamental flaw in utilizing the baseline SBFC model to analyze real-world social networks is its assumption of a homogeneous population. In the baseline model, every node in the graph exhibits the exact same propensity to verify information ( $p_{\text{verify}}$ ), the exact same rate of memory decay ( $p_{\text{forget}}$ ), and equal susceptibility to peer influence. Real digital social ecosystems, however, are characterized by vast disparities in user influence, media literacy, engagement algorithms, and automated bot activity.

The model selected for this HPC performance analysis addresses these limitations by introducing four distinct, behaviorally heterogeneous classes of agents. This extension pushes the model closer to reality and simultaneously increases the algorithmic complexity of the simulation, making it an excellent candidate for benchmarking distributed computing systems.

This enhancement transforms the localized neighbor counting mechanism. The transition equations that calculate  $n_i^B(t)$  and  $n_i^F(t)$  must now aggregate the influences of neighbours separated by their respective classes, requiring the simulation to query not only the state of each neighbor, but also its inherent class type during every iteration:

$$n_i^B(t) = n_{\text{SC}_i}^B(t) + n_{\text{I}_i}^B(t) + n_{\text{N}_i}^B(t) + n_{\text{BOT}_i}^B(t) \quad (8)$$

$$n_i^F(t) = n_{\text{SC}_i}^F(t) + n_{\text{I}_i}^F(t) + n_{\text{N}_i}^F(t) + n_{\text{BOT}_i}^F(t) \quad (9)$$

where the subscripts SC, I, N, and BOT denote Scholars, Influencers, Normal agents, and Bots, respectively. Follows a description of each agent type characteristics, summarized in Table 2:

- **Normal agents:** Normal agents constitute the vast, overwhelming majority of the network graph. They represent the standard, everyday users of a social media platform who consume and share information without specialized agendas or elevated media literacy. In the simulation, the Normal class operates strictly according to the baseline parameters originally defined in the homogeneous SBFC model. Specifically, they exhibit a standard, relatively high forgetting probability ( $p_{\text{forget}} = 0.1$ ) and a low baseline verification probability ( $p_{\text{verify}} = 0.05$ ). Because they represent the numerical bulk of the population, the global macroscopic state of the simulation, that is whether the hoax ultimately becomes an endemic fixture of the network or is successfully eradicated, heavily depends on the shifting beliefs of this specific class. Their behavior provides the underlying substrate through which the other, more specialized classes operate.
- **Scholars:** The Scholar class represents a minority subset of users characterized by high media literacy, a deeper understanding of factual information, and a proactive approach to debunking falsehoods. Behaviorally, Scholars exhibit a substantially higher probability of independently verifying incoming news, with experimental configurations testing  $p_{\text{verify-scholar}} \in [0.05, 0.3]$ . Furthermore, because their beliefs are rooted in verified evidence rather than fleeting trends, they exhibit a significantly lower probability of forgetting their verified stance, with  $p_{\text{forget-scholar}} \in [0.02, 0.1]$ .

A critical topological constraint distinguishes the Scholar class from the rest of the population: they are not randomly or uniformly distributed across the network graph. Instead, they are densely concentrated within specific, highly interconnected structural clusters, simulating real-world academic, scientific, or professional knowledge silos.

- **Influencers:** Influencers model human users who possess disproportionate social reach and exceptionally high network centrality. Topologically, they are rigidly defined as the top 1% of nodes sorted by out-degree connectivity. Because they are connected to a massive number of follower neighbours, any state change in an Influencer rapidly cascades through the network, creating massive localized ripples in the  $f_i(t)$  and  $g_i(t)$  functions of thousands of normal agents. Behaviorally, Influencers are modeled to hold onto their public beliefs longer than Normal agents to maintain their brand identity, resulting in a lower  $p_{\text{forget}}$ . They also possess a variable, but generally higher, verification probability than normal users, simulating the resources they might have to evaluate information before broadcasting it to their large audiences.
- **Bots:** The Bot class introduces fully automated, algorithmic, and static agents into the social ecosystem. Bots represent the top 10% of nodes in terms of degree connectivity, explicitly excluding the already designated human Influencers. Crucially, Bots do not participate in the cognitive Markov transitions of the SBFC framework. They possess a zero probability of forgetting ( $p_{\text{forget}} = 0$ ) and a zero probability of verification ( $p_{\text{verify}} = 0$ ). Once initialized at  $t = 0$ , their state is permanently immutable. Bots are further subdivided into two adversarial factions: Believer Bots and Fact-Checker Bots.

### 2.1.3 Experimental Input Parameters

Evaluating the performance of this model within an HPC framework requires a rigorous, systematic parameter sweep. The experimental space must deliberately push the simulation across varied phases of execution: from states where the hoax dies out rapidly (resulting in low computational interaction and early equilibrium) to states of intense, enduring, network-wide competition between Believer and Fact-Checker cascades (resulting in high computational interaction and maximum resource utilization).

The defined input variables and their specific tested values, derived from the original literature, are detailed in Table 3.

|                   | Topology                    | $p_{\text{verify}}$ | $p_{\text{forget}}$ |
|-------------------|-----------------------------|---------------------|---------------------|
| <b>Normal</b>     | Uniformly distributed       | 0.05                | 0.1                 |
| <b>Scholar</b>    | Densely concentrated        | $\in [0.05, 0.3]$   | $\in [0.02, 0.1]$   |
| <b>Influencer</b> | Nodes with top 1% of edges  | $\in [0.05, 0.2]$   | $\in [0.02, 0.1]$   |
| <b>Bot</b>        | Nodes with top 10% of edges | 0.0                 | 0.0                 |

Table 2: Agent type characteristics of the heterogeneous SBFC model



| Param                           | Description                                                                                                                                                  | Values                      |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| $\alpha$                        | The inherent believability factor of the hoax. Higher values artificially increase the mathematical weight of Believer neighbors in the spreading functions. | $\{ 0.3, 0.8 \}$            |
| $\beta$                         | The base transmission intensity determining how quickly beliefs spread from neighbors to a Susceptible agent, independent of truth.                          | $\{ 0.5, 0.75 \}$           |
| $B_0$                           | The percentage of the total agent population initialized in the Believer state at $t = 0$ , setting the initial infection seed.                              | $\{ 10\%, 40\% \}$          |
| $p_{\text{verify}}^{\text{SC}}$ | The specific probability that an agent in the Scholar class will independently verify the news.                                                              | $\{ 0.05, 0.1, 0.2, 0.3 \}$ |
| $p_{\text{forget}}^{\text{SC}}$ | The specific probability that a Scholar agent will forget their belief and revert to Susceptible.                                                            | $\{ 0.02, 0.05, 0.1 \}$     |
| $p_{\text{verify}}^{\text{I}}$  | The specific probability that an agent in the Influencer class will independently verify the news.                                                           | $\{ 0.05, 0.1, 0.2 \}$      |
| $p_{\text{forget}}^{\text{I}}$  | The specific probability that an Influencer will forget their belief and revert to Susceptible.                                                              | $\{ 0.02, 0.05, 0.1 \}$     |
| $\text{BOT}_B$                  | The percentage of top-degree nodes (excluding Influencers) algorithmically converted into static Believer Bots.                                              | $\{ 0\%, 1\%, 2\%, 5\% \}$  |
| $\text{BOT}_F$                  | The percentage of top-degree nodes (excluding Influencers) algorithmically converted into static Fact-Checker Bots.                                          | $\{ 0\%, 1\%, 2\%, 5\% \}$  |
| Ticks                           | The temporal duration of the simulation. Each tick represents one hour. 168 ticks represent a 7-days lifecycle.                                              | 168                         |

Table 3: Input parameters of the heterogeneous SBFC model

### 2.1.4 Validation Methodology

To verify that the newly developed C++ Repast HPC implementation correctly captures the intricate dynamics of the original model, a rigorous validation protocol must be established. Because the underlying transition functions are highly stochastic, direct 1:1 validation of a single run is mathematically impossible; instead, the validation framework must rely on demonstrating statistical equivalence over large sample sizes.

The primary metrics for validation will be the mean densities of the three agent states  $B_\infty$ ,  $\text{FC}_\infty$ , and  $S_\infty$ , at the conclusion of the 168-tick simulation cycle. The baseline experiments in the original literature generated 4 replicates for each of the 13,824 settings. Analysis of these replicates revealed a standard deviation of less than 5% relative to the total agent population, indicating exceptionally high macroscopic model stability despite individual micro-level stochasticity.

The Repast HPC model will be executed using identical input parameter matrices and identical network topology. The resulting macroscopic curves must statistically match the baselines within a strict 95% confidence interval. For example, the explosive,

non-linear growth of Believers when hoax credibility is maximized ( $\alpha = 0.8$ ), versus the total systemic dominance of Fact-Checkers when hoax credibility is minimized ( $\alpha = 0.3$ ), must be perfectly replicated. Additionally, the localized, systemic effects of the heterogeneous classes, such as the specific 8-9% reduction in the global Believer population when the Scholar class is properly configured and educated, must be demonstrably reproducible in the parallelized environment.

Only once this statistical parity is definitively achieved can the true performance analysis phase commence. The benchmark suite will then systematically scale the agent population and the allocated MPI processes, rigorously recording total execution time, memory overhead, parallel efficiency, and MPI communication latency to fully expose, analyze, and document the architectural bottlenecks of distributing complex social network models across modern High Performance Computing environments.

## Chapter 3

# Model implementations

The core objective of the upcoming third chapter of this Master’s thesis is the adaptation and implementation of the aforementioned social network models onto the Repast HPC platform. Transitioning the models from a single-threaded, high-level, interpreted environment like NetLogo into a highly distributed, compiled C++ and MPI-based framework requires profound architectural shifts and a deep understanding of parallel computing paradigms. The implementations must carefully map the conceptual socio-cognitive components of the ABM’s to Repast HPC’s strict distributed memory architecture.

### 3.1 Parallelizing the misinformation diffusion model

Translating the extended SBFC model from its original NetLogo format into Repast HPC demands a fundamental shift in programming paradigms. While NetLogo relies on an interpreted, single-threaded execution loop that evaluates agents sequentially or in randomized batches, Repast HPC utilizes a compiled, distributed-memory C++ architecture built on top of the Message Passing Interface (MPI).

In a distributed environment, the global network of 4,039 Facebook nodes is not held in a single block of memory. Instead, the network graph and its constituent agents must be spatially partitioned across multiple processes (ranks). As the simulation scales to millions of agents in later benchmarking phases, the algorithms governing agent interaction, specifically the spreading functions evaluating neighbor states, must be carefully constructed to accommodate data that is non-local and potentially stale between synchronization steps.

#### **Representing the misinformation diffusion agent in RepastHPC**

In Repast HPC, the foundational memory container for the simulation is the **Context**. The **Context** encapsulates the agent population, ensuring through strict set semantics that only a single, unique instance of an agent exists within the logical bounds of the simulation. For this misinformation model, the **Context** will hold all instances of Scholars, Influencers, Normal agents, and Bots, managing their creation and lifecycle.

To represent the complex Facebook topology used in the original model simulations, the model must implement a **SharedNetwork** projection. A projection in Repast HPC is a construct that imposes a specific relational structure upon the otherwise unstructured pool of agents residing in the **Context**. When an agent is instantiated and added to the **Context**, it automatically becomes a vertex in the **Network** projection, and the specific edges representing social connections are mapped according to the parsed dataset.

The temporal execution of the model is governed by Repast HPC’s dynamic discrete-event scheduler. Unlike simple loop-based simulators, the scheduler operates on a principle of conservative synchronization. At each tick, representing one hour of simulated time in the news cycle, the scheduler pops the next prioritized event from its queue. In the context of the misinformation model, a single scheduled tick involves a massive sequence of highly orchestrated operations for every active agent:

1. Query the current state (S, B, FC) and the specific class type (SC, I, N, BOT) of all connected neighbours via the **Network** projection.
2. Calculate the fractional ratios for the spreading functions  $f_i(t)$  and  $g_i(t)$ .
3. Evaluate the independent verification ( $p_{\text{verify}}$ ) and forgetting ( $p_{\text{forget}}$ ) probabilities, dynamically accessing the specific parameters based on the agent’s assigned class.
4. Generate a high-entropy random number using the boost/MPI synchronized random number generators to trigger the stochastic transitions.
5. Determine and lock in the discrete state for  $t + 1$ .

### Distributed memory management

Repast HPC is explicitly designed for distributed-memory hardware architectures, utilizing the Message Passing Interface (MPI) standard to parallelize operations across multiple separate compute nodes. In this architecture, the global **Context** and the massive **Network** projection do not exist as a single, contiguous entity in any single processor’s memory. Instead, the agents are spatially partitioned and distributed across multiple MPI processes, known as **ranks**.

Each MPI process is strictly and exclusively responsible for the computation of its assigned local agents. For example, if Process 1 (P1) holds Agent A, and Process 2 (P2) holds Agent B, P1 computes the transition logic for A, and P2 independently computes it for B. However, the highly interconnected nature of a scale-free social network guarantees that thousands of edges will span across process boundaries. If Agent A (residing on P1) is followers with Agent B (residing on P2), Agent A absolutely needs to read Agent B’s state to correctly calculate its local spreading functions  $f_A(t)$  and  $g_A(t)$ .

Because physical memory is not shared in an MPI environment, P1 cannot simply dereference a pointer to read the memory address of Agent B on P2. Repast HPC resolves this fundamental distributed computing problem through the creation of “non-local” or “ghost” agents. In this system, P1 requests a copy of Agent B from P2 over the network. This ghost copy is instantiated and resides locally in P1’s **Context**. When Agent A iterates through its neighbours during the update step, it reads the local ghost copy of Agent B without needing to perform an expensive network call mid-computation.

### Parallel communication management

Crucially, the code executing on P1 is strictly forbidden from altering or writing to the state of the ghost Agent B. Agent B is the sovereign property of its home process, P2. Because Agent B might stochastically change its state from Susceptible to Believer on P2 during a tick, the ghost copy sitting on P1 becomes instantly stale.

To maintain strict simulation coherence and prevent causality errors, Repast HPC requires the explicit, synchronized updating of all ghost agents across all process boundaries at the end of every tick. This complex data serialization is achieved using the Repast HPC **Package** pattern. The C++ implementation of the model must define:

- A **Package**: A lightweight, optimized C++ struct containing only the absolute minimal state data required for network transmission, in this case, the agent’s unique ID, its current belief state, and its class type.
- A **provider**: A routine on the home process (P2) that efficiently extracts the state from the real Agent B and serializes it into the **Package** for outbound MPI transmission.
- A **receiver**: A routine on the requesting process (P1) that receives the inbound **Package** array, deserializes the data, and maps the updates to the corresponding ghost Agent B.

The implementation will utilize Repast HPC’s default **GhostMode** synchronization protocol. In **GhostMode**, writes are strictly local, and distant agents are read from a ghost copy representing the stable system state at the previous time step ( $t - 1$ ). While this ensures strict determinism and eliminates read-write race conditions, the mandatory block-synchronization of the entire boundary layer at the end of the tick induces massive latency, forming the primary barrier to scalability.

Figure 2 depicts how Repast HPC handles the distributed memory and parallel communication in the specific case of the heterogeneous SBFC model: The global network that holds the relationships between agents is partitioned to distribute the work load among the multiple processes or ranks. To read data from agents in foreign processes, the framework requests a ghost read-only copy to the provider process and hands it over to the receiver process.

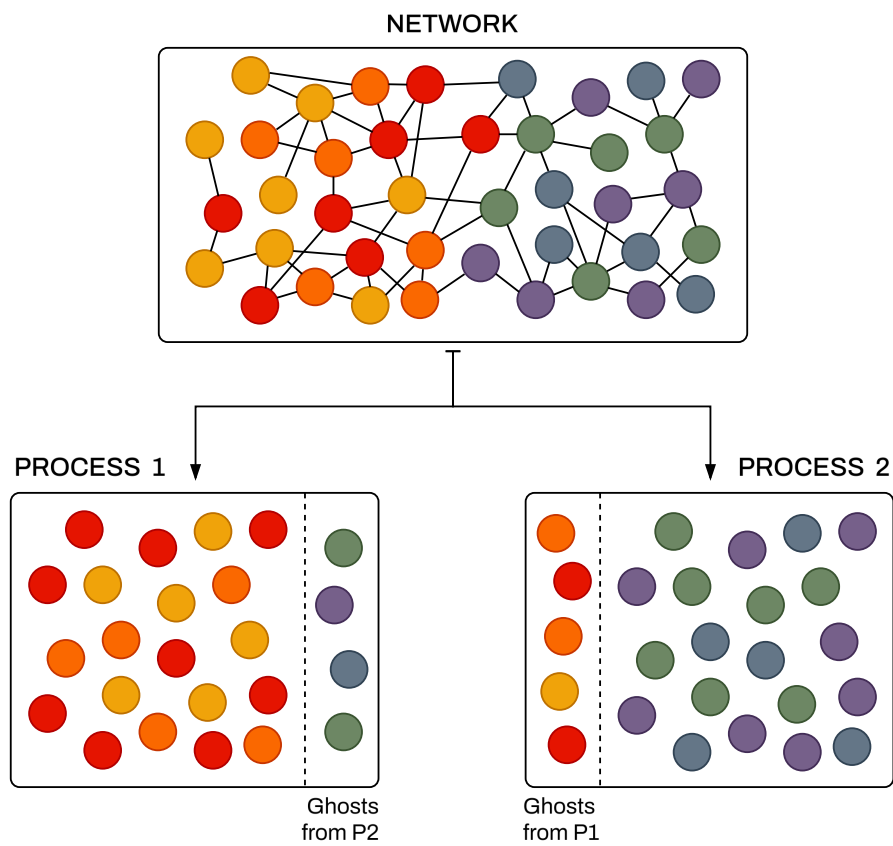


Figure 2: Repast HPC memory management applied to the SBFC model

## Bibliography

- [1] S. Coakley, M. Gheorghe, M. Holcombe, S. Chin, D. Worth, and C. Greenough, "Exploitation of high performance computing in the FLAME agent-based simulation framework," 2012, pp. 538–545. doi: 10.1109/HPCC.2012.79.
- [2] P. Richmond, S. Coakley, and D. M. Romano, "A high performance agent based modelling framework on graphics card hardware with CUDA," in *Proceedings of The 8th International Conference on Autonomous Agents and Multiagent Systems-Volume 2*, 2009, pp. 1125–1126.
- [3] N. T. Collier and M. J. North, "Parallel Agent-Based Simulation with Repast for High Performance Computing," *Simulation*, vol. 89, no. 10, pp. 1215–1235, 2013, doi: 10.1177/0037549712462620.
- [4] Repast Development Team, "RepastHPC: A High-Performance Agent-Based Modeling Platform." [Online]. Available: <https://repast.github.io/>
- [5] G. Cordasco, R. D. Chiara, A. Mancuso, D. Mazzeo, V. Scarano, and C. Spagnuolo, "Bringing together efficiency and effectiveness in distributed simulations: the experience with D-MASON," *Simulation*, vol. 89, no. 10, pp. 1236–1253, 2013, doi: 10.1177/0037549713493078.
- [6] A. Rousset, B. Herrmann, C. Lang, and L. Philippe, "A survey on parallel and distributed multi-agent systems for high performance computing simulations," *Computer Science Review*, vol. 22, pp. 27–46, 2016, doi: 10.1016/j.cosrev.2016.08.001.
- [7] A. Moreno, J. Jorba, C. Peralta, E. César, A. Sikora, and M. Hanzich, "A methodology for selecting a performance-convenient ABMS development framework on HPC platforms," *Simulation Modelling Practice and Theory*, 2023, doi: 10.1016/j.simpat.2023.102812.
- [8] N. D. Rollins, C. M. Barton, S. Bergin, M. A. Janssen, and A. Lee, "A Computational Model Library for Publishing Model Documentation and Code," *Environmental Modelling & Software*, vol. 61, pp. 59–64, 2014, doi: 10.1016/j.envsoft.2014.06.022.
- [9] CoMSES Net Community, "CoMSES Net Computational Model Library." [Online]. Available: <https://www.comses.net/codebases/>
- [10] A. Moreno, J. J. Rodriguez, D. Beltran, A. Sikora, J. Jorba, and E. Cesar, "Designing a benchmark for the performance evaluation of agent-based simulation applications on HPC," *The Journal of Supercomputing*, vol. 75, pp. 1524–1550, 2019, doi: 10.1007/s11227-018-2688-8.
- [11] A. K. Yalabadi, M. Yazdani-Jahromi, S. Abdizaji, I. Garibay, and O. O. Garibay, "Controlling the Misinformation Diffusion in Social Media by the Effect of Different Classes of Agents," 2024, doi: 10.48550/arXiv.2401.11524.
- [12] A. K. Yalabadi, "Controlling the misinformation diffusion in social media by the effect of different classes of agents, Version 1.0.0." [Online]. Available: <https://www.comses.net/codebases/48df051c-1f3a-4a27-94d0-bf2253947816/releases/1.0.0/>
- [13] M. Tambuscio, G. Ruffo, A. Flammini, and F. Menczer, "Fact-checking Effect on Viral Hoaxes: A Model of Misinformation Spread in Social Networks," in *Proceedings of the 24th International Conference on World Wide Web Companion (WWW '15 Companion)*, New York, NY, USA: ACM, 2015, pp. 977–982. doi: 10.1145/2740908.2742572.
- [14] E. Sulis and M. Tambuscio, "Simulation of misinformation spreading processes in social networks: an application with NetLogo," in *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*, 2020, pp. 614–618. doi: 10.1109/DSAA49011.2020.00086.
- [15] A. Willmore, "This Analysis Shows How Viral Fake Election News Stories Outperformed Real News on Facebook." [Online]. Available: <https://www.buzzfeednews.com/article/craigsilverman/viral-fake-election-news-outperformed-real-news-on-facebook>